Практическая работа №5
Тема: Предметно-ориентированное проектирование

Введение

Зачем нужно предметно-ориентированное проектирование и где его применять? Чтобы ответить на этот вопрос нужно понимать, что предметно-ориентированное проектирование есть не сама архитектура и заключенные в нее важные базисы, а и стиль разработки программного обеспечения, это методология не привязанная к какой-либо архитектуре. DDD - это совокупность передовых методик, которые используются при разработке программного обеспечения.

Зачем использовать DDD если можно разрабатывать продукт, например, с использованием технологии Scrum или любой другой для менеджмента проекта, использовать архитектуру, которая решает часть проблем. Чтобы ответить на этот вопрос нужно понимать каких проблем получается избежать или свести их к минимум при использовании методологии предметно-ориентированного проектирования:

1. Разработка программного обеспечения считается статьей расходов для компании. Данная проблема относится к ряду бизнес-проблем. Для решения данной проблемы следует рассматривать программное обеспечение, как стратегическое преимущество, но не все компании морально готовы к данному шагу.
2. Разработчики пытаются решать проблемы используя технологии. Данная проблема относится к техническому спектру проблем и довольна интересна и на первый взгляд не выглядит проблемой, но погоня за самыми современными технологиями в конце концов окажет сильное негативное влияние, в отличие от трезвого расчета и продуманного дизайна(архитектуры).
3. На первом плане находятся модели данных и база данных для их хранения,а не бизнес-процессы и операции. Вспоминая архитектурные принципы Роберта Мартина. База данных является деталью, на которую следует обращать внимание в последнюю очередь, ведь конечная организация хранения данных является второстепенным вопросом, а бизнес-правила имеют высший приоритет.
4. Пропасть между программных обеспечением и бизнесом, а точнее бизнес-моделью из-за того, что разработчики ПО применяют другое наименование объектов и операций, не как в бизнес-модели, что влечет путаницу на уровне взаимодействия разработчиков программного обеспечения и представителей бизнеса.
5. Ориентированность на проектные сметы в итоге превращает проект в Большой ком грязи, что не является чем-то хорошим в будущем. Про большой ком грязи будет сказано далее в лекции.
6. Нарушается принцип разделения политики и деталей и происходит смешение бизнес-логики, пользовательского интерфейса и других компонентов системы.
7. Чрезмерное обобщение решений без привязки к бизнес-потребностям, что возвращает к проблеме плохого взаимодействия бизнеса и разработчиков программного обеспечения.

Что дает хороший дизайн? Хороший дизайн является эффективным дизайном. Он удовлетворяет потребности организации, что позволяет получить требуемое и желаемое стратегическое преимущество благодаря продвинутому и эффективному программного обеспечению.

Моделирование системы нужно, даже необходимо, отказ от него - есть ложная идея сэкономить, вследствие неправильных бизнес-решений и далее терпеть неудачи одну за другой во всех процессах жизненного цикла программного обеспечения.

Стратегическое проектирование

Говоря и думая о стратегическом проектировании, должна представляться картина, написанная крупными мазками. Это выделение стратегически важных аспектов бизнеса, разделение работы по степени важности и объединении всего при необходимости. Оно предполагает выделение предметной области(Domain model) с помощью шаблона стратегического проектирования ограниченный контекст(bounded context), а также разработку единого языка(ubiquitous language) в рамках модели предметной области в пределах определенного ограниченного контекста. Для схематичного представления на рисунке 1 приведен ограниченным контекст и единый язык.

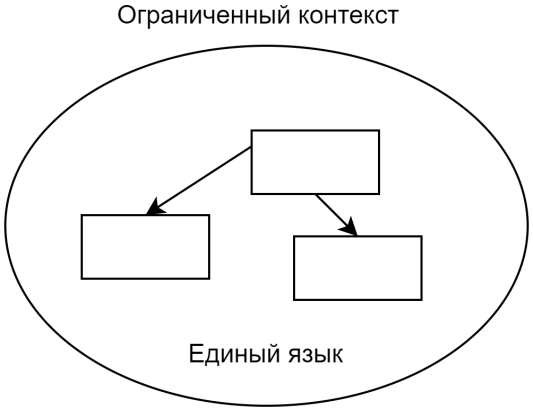


Рисунок 1 - Ограниченный контекст и единый язык

Ограниченный контекст — это семантическая контекстная граница, что значит в пределах ограниченного контекста каждый компонент программного обеспечения имеет определенное значение и делает определенные вещи. Компоненты внутри ограниченного контекста являются характерными для данного контекста и семантически обоснованными. Также в рамках моделирования, ограниченный контекст - это область, в которой реализуется модель, и каждому ограниченному контексту соответствует отдельный артефакт программного обеспечения.

При начале разработки, когда ограниченный контекст носит теоретический характер, то его можно отнести к пространству задач, а когда идет активная разработка, то к пространству решений. Пространство задач — это область, в которой осуществляется стратегический анализ высокого уровня и выполняются этапы проектирования с учетом заданных ограничений. При необходимости в ходе обсуждения можно использовать ограниченный контекст. Пространство решений — это область , в которой фактически реализуется решение, идентифицированное в результате анализа пространства задач как Смысловое ядро(core domain).

Смысловым ядром называется ограниченный контекст, который разрабатывается как ключевая стратегическая инициатива организации. Решение в данном контексте преобразуется в исходный код, кроме того в пространстве решений разрабатывается код, для интеграции с другими ограничительными контекстами. Это самая важная программная модель и разрабатывается так, чтобы организация могла конкурировать со всеми остальными компаниями. Смысловое ядро должно решать основные проблемы бизнеса. Поэтому первое ценностное предложение DDD - выбрать смысловое ядро и выделить для него лучшие ресурсы.

Единым же языком называют язык, отражающий программную модель, функционирующую в ограниченном контексте. Он используется как для общения разработчиков, так и для реализации модели. Разработчики и другие члены бизнеса говорят на этом языке и все в итоге друг друга понимают. При этом ограниченный контекст можно сравнить с языковыми границами. Для DDD речь идет о языках, на которых говорят члены группы разработчиков

программной модели, а также языках программирования и других технологиях, на которых реализован исходный код.
Существует правило - один ограниченный контекст, одна группа его разрабатывающая, один репозиторий исходного кода. При этом одна группа может работать над несколькими ограниченными контекстами, но несколько групп над одним работать не могут. Это является еще одним преимуществом DDD - без ведома разрабатывающей группы ничего не произойдет в исходном коде необратимого.
Весомой причиной использования ограниченных контекстов является большой ком грязи, в который превращается программное обеспечение, когда зашкаливает количество концепций и модель становится неуправляемой. большой ком грязи — это результат попытки сделать слишком много, со слишком многими людьми и в неправильном месте.
Какие инструменты DDD позволяют избежать ситуации Большого кома грязи? Для этого нужно как минимум два инструмента фундаментального стратегического проектирования: ограниченный контекст и единый язык. Использование ограниченного контекста предполагает объединение близких концепций, которые являются ядром стратегической инициативы, и отбрасывать все остальное.
Что же такое ядро? Ядро — это объединение экспертов предметной области и разработчиков программного обеспечения.
Основой же стратегического проектирования является не только разграничение на ограниченные контексты и выделение смыслового ядра, но и разработка единого языка.
Самое важное преимущество и характерная особенность DDD заключается в том, что реально можно обсуждать модели предметной области, то есть ее дизайн. Но важно помнить о том, чтобы обсуждение не уходило слишком далеко в диаграммы и рисунки, ведь модель есть код и код есть модель.
Существует вопрос о том, что находиться в ограниченном контексте. В ограниченный контекст входит не только модель предметной области. Ограниченный контекст делится на несколько основных уровней: адаптеры ввода, службы приложений, оркестрирующие сценарии использования и управляющие транзакциями, модуль предметной области, на которой было исключительное сосредоточение в данном разделе, а также адаптеры вывода. Схематично данные уровни изображены на рисунке 2. Самое главное, что нужно понимать, что модель предметной области должна быть свободна от технологии.



Рисунок 2 - уровни архитектуры DDD

В адаптере ввода находится безопасность, пользовательский интерфейс и представления, а в адаптере вывода репозитории, документы, кеш, рассылка сообщений. В свою очередь служба приложения содержит также безопасность, транзакции, координацию задач и контроллер сценариев использования. А модель предметной области содержит сущности, бизнес-логику и события предметной области.
Порты и адаптеры можно использовать как базовую архитектуру, но это не единственная архитектура, которую можно использовать с подходом DDD. Можно использовать следующие архитектурные походы и паттерны:

1. Архитектура, управляемая событиями: источники событий.
2. Разделение ответственности на запросы и команды.

3. Реактивная модель и модель актера.
4. Передача состояния представления.
5. Сервис-ориентированная архитектура.
6. Микрослужбы.
7. Облачные вычисления.

При проектировании на основе методологии DDD возникает ограниченный контекст, содержащий смысловое ядро, и другие вспомогательные ограниченные контексты. То, что содержится в данных контекстах называется подобластями. Подобласть — это часть предметной области. Подобласти можно использовать для логического разделения предметной области, чтобы понять пространство состояния большого и сложного объекта. Другой способ интерпретации подобласти состоит в том, что она рассматривается как четко очерченная область компетенции, позволяющая найти решения для смыслового ядра бизнеса. Это означает, что в конкретной подобласти есть эксперты предметной области, которые хорошо понимают аспекты бизнеса, ведение которого облегчает данная область.
В рамках проекта существует три типа подобластей: смысловое ядро, вспомогательная подобласть и универсальная подобласть. Вспомогательная подобласть требует специального моделирования ситуаций, для которых не существует готовых решений. Разработку ограниченных контекстов таких областей можно поручить сторонним организациям и не вкладывать основные инвестиции в данную подобласть в отличие от смыслового ядра. Универсальная подобласть в свою очередь это вид решения, которое можно приобрести уже готовым, или привлечь к разработке сторонние организации.
Все понятия, которые не принадлежат смысловому ядру, относятся к одному из ограниченных контекстов. Важно понимать, что существует некоторое взаимодействие, оно даже необходимо, смыслового ядра в виде ядра управления гибким проектированием и другими ограниченными контекстами. Эта интеграция в DDD называется связыванием контекстов. Вопрос в том, какие существуют способы связывания контекстов.
Между двумя группами может существовать отношения партнерства. При этом каждая из групп является ограниченным контекстом. Партнерство преследует цель согласования целей двух групп. Схематично данное связывание контекстов изображено на рисунке 3:

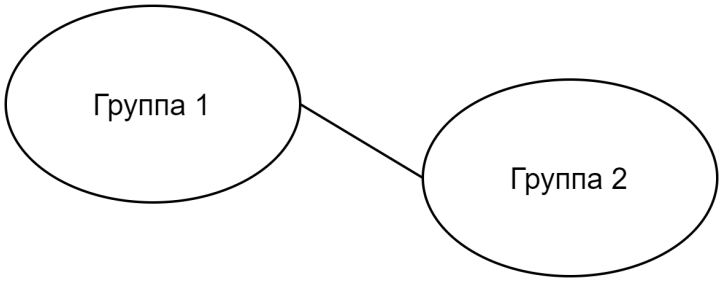


Рисунок 3 -связывание контекстов отношение партнерства

На данном уровне идет тесное взаимодействие двух групп разработки определенное количество времени, так как долгое взаимодействие и взаимосвязь тяжеловесны и практически невозможны.

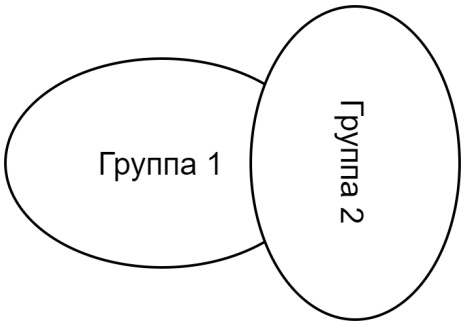


Рисунок 4 - связывание контекстов отношение общее ядро

Вторым типом связывания контекстов является общее ядро. Данное связывание образуется путем пересечения двух ограниченных контекстов. Данное отношение предполагает использование несколькими группами одной общей маленькой модели. Данное связывание изображено на рисунке 4. Общее ядро трудно выделить и обслуживать из-за постоянного требуемого согласования.
Третьим способом является связывание клиент-поставщик. Отношение клиент-поставщик описывает взаимосвязи между двумя ограниченными контекстами и соответствующими группами, где поставщик - это вышестоящий контекст(обозначается буквой U на диаграмме),а клиент нижестоящий контекст(обозначается буквой D на диаграмме). Поставщик обеспечивает то, в чем нуждается клиент. Клиент должен в свою очередь планировать взаимодействие с поставщиком, поставщик определяет то, что получит клиент. Данное отношение изображено на рисунке 5:

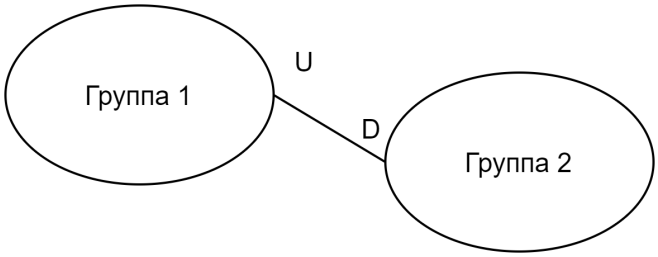


Рисунок 5 - отношение поставщик-клиент и конформист

Еще одним отношением является отношение конформист. Оно возникает, когда есть вышестоящая и нижестоящая группы. И вышестоящая не имеет никакого стимула для определения потребностей нижестоящей группы. То есть нижестоящая группа подстраивается под вышестоящую модель и называется конформистом. Данное отношение изображено на рисунке 5.

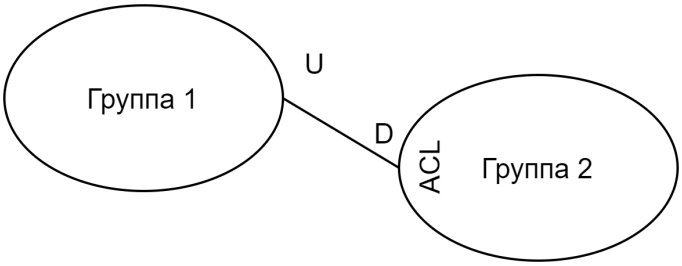


Рисунок 6 - Отношение конформист

Еще одним отношением является отношение предохранительный уровень. Это отношение связывания контекстов, обеспечивающее их защиту. В этом отношении нижестоящая группа создает уровень перевода между своей моделью и единым языком(моделью) вышестоящей группы. Этот уровень является неким барьером и обеспечивает обмен сообщениями между моделями. Таким образом, это отношение является разновидностью интеграции и изображено на рисунке 6.
Еще одним отношением является служба с открытым протоколом, изображенная на рисунке 7.

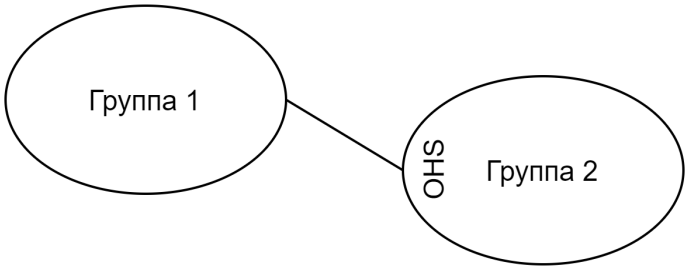


Рисунок 7 - служба с открытым протоколом

Данная служба предоставляет протокол или интерфейс, предоставляющий доступ к ограниченному контексту с помощью ряда служб.
Общедоступный язык - это хорошо задокументированный язык информационного обмена, допускающий простое употребление и перевод для любого количества ограниченных контекстов. Отношения с использованием общедоступного языка изображено на рисунке 8.

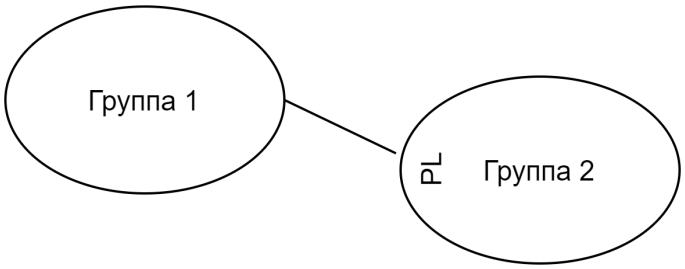


Рисунок 8 - отношение с общедоступным языком

Потребители, читающие и пишущие на общедоступном языке, могут с него осуществлять прямой и обратный перевод. Часто службы с открытым протоколом поддерживают и используют общедоступный язык, чтобы обеспечивать лучшую интеграцию для третьих сторон. Эта комбинация делает очень удобным перевод между двумя едиными языками.
Еще одним отношением является отношение отдельного существования. Отдельное существование описывает ситуацию, в которой интеграция с одним или несколькими ограниченными контекстами не приносит выгоды за счет использования разных единых языков. Данное отношение изображено на рисунке 9.

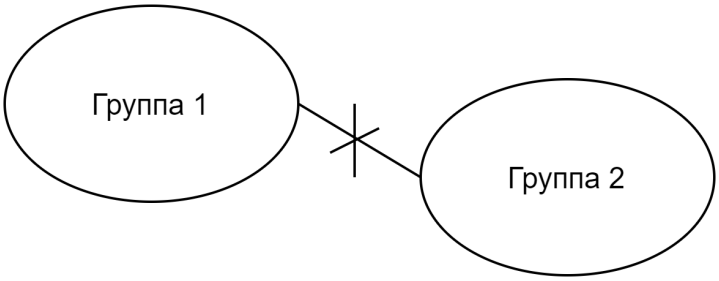


Рисунок 9 - отдельное существование

Тактическое проектирование

Стратегическое проектирование предполагает создание модели с помощью ограниченных контекстов, подобластей и карт контекстов, образованных ограниченными контекстами и отношениями между ними. Что можно сказать о концепциях, которые находятся в ограниченном контексте? Пример изображен на рисунке 10.

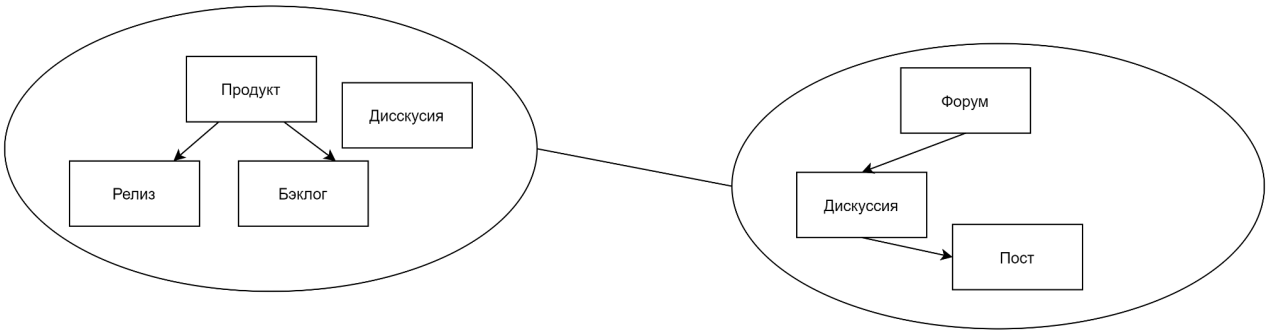


Рисунок 10 - смысловое ядро и вспомогательная подобласть

Каждая из концепций, изображенная внутри ограниченных контекстов, является агрегатом. Кроме той, что не имеет отношений внутри ограниченного контекста. В случае рисунка 10 — это Дискуссия в ограниченном контексте слева. Дискуссия является объектом-значение.
Что такое сущность? Сущность моделирует индивидуальный объект. Каждая сущность имеет уникальную идентичность, которая отличает ее от всех остальных сущностей то же самого или другого типа.

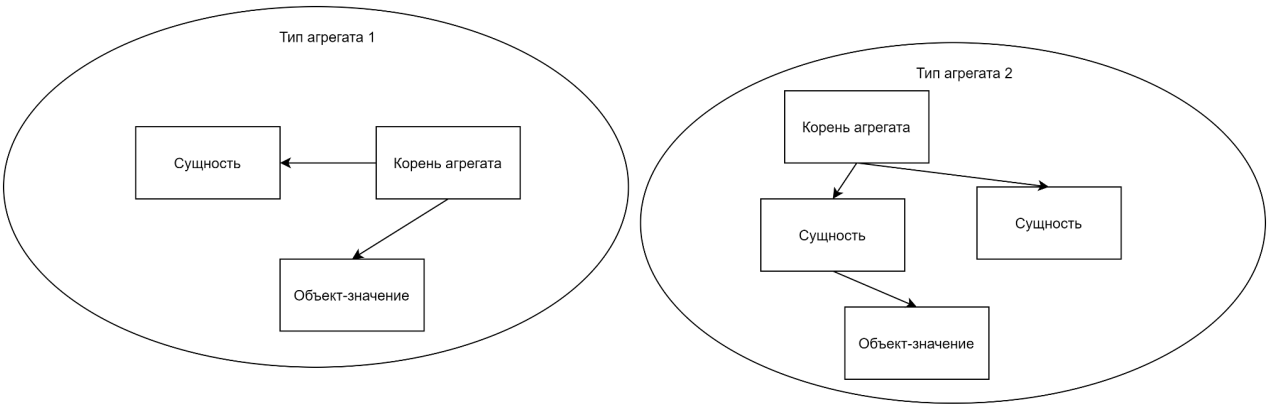


Рисунок 11 - сущности, агрегаты и объекты-значения

На рисунке 11 представлены два типа агрегатов. Каждый агрегат состоит из одной и более сущностей, одна из которых обязательно является корнем агрегата. Кроме того, агрегаты могут содержать объекты-значения.
Объект-значение или просто значение модулирует неизменное концептуально цельное понятие. Пределах модели концепция представляет собой значение. Отличие от сущности, значение не имеет уникальной идентичности, а эквивалентность значений определяется с помощью сравнения атрибутов, инкапсулированных в виде значения. Важно понимать, что значение — это не предмет, а средство определения количества или измерения сущности.
Корневая сущность каждого агрегата владеет всеми остальными элементами, собранными в агрегате. Это проиллюстрировано на рисунке 11. Именем корневой сущности является концептуальное имя агрегата. Имя выбирает под стать концепции моделирования определенного агрегата.
Каждый агрегат формирует границу согласованности транзакций. Это значит, что, когда управляющая транзакция передается в базу данных, все составные части в пределах отдельного агрегата должны быть непротиворечивыми и соответствовать бизнес-правилам. Границы транзакции определяются бизнесом, потому что именно он определяет, какое состояние кластера является правильным в любой заданный момент момент. Правилом транзакции является то, что в одной транзакции можно модифицировать и фиксировать только один экземпляр агрегата. Любой агрегат будет изменяться и фиксироваться в другой транзакции.

В чем состоят другие эмпирические правила проектирования агрегатов? Первым правилом является защита бизнес-инвариантов в границах агрегата. Это значит, что в итоге бизнес определяет состав агрегата, основываясь на том, какие концепции должны оставаться согласованными после совершения транзакции. Для примера рассмотрим рисунок 12.

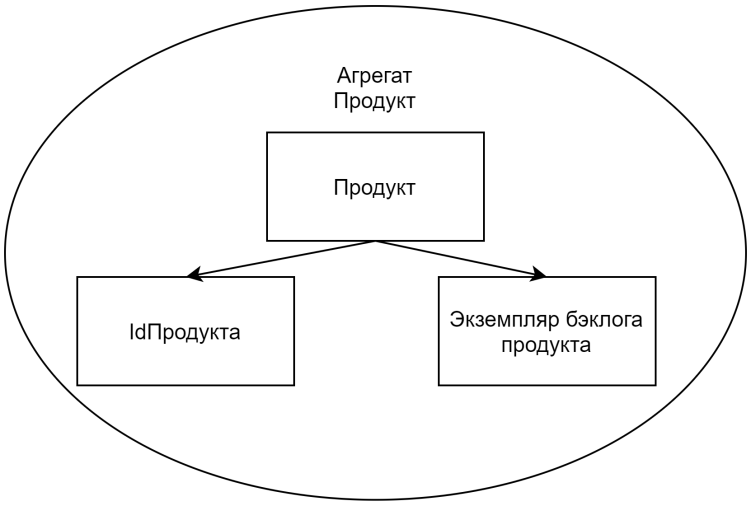


Рисунок 12 - Агрегат Продукт

Класс Продукт проектируется так, что в конце транзакции все образованные экземпляры Экземпляр бэклога продукта должны быть совместимыми и согласованными с корнем класса Продукт.
Вторым правилом является рекомендация проектировать маленькие агрегаты. Это правило подчеркивает, что каждый агрегат должен занимать небольшую область оперативной памяти и иметь небольшую область видимости транзакции. Следование этому правилу дает дополнительную выгоду в том, что с каждым агрегатом будет проще работать, потому что каждая связанная задача может управляться отдельными разработчиками. Это также означает, что агрегат будет проще для тестирования.
Проектируя агрегаты, следует помнить о принципе единственной обязанности. Если агрегат делает слишком многое, то это нарушает принципе единственной обязанности. Второе правило изображено на рисунке 13.

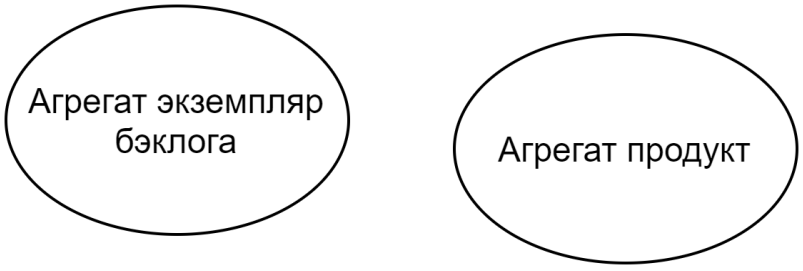


Рисунок 13 Разделение агрегатов

Третьим правилом является то, что ссылаться на агрегаты только по их идентификаторам. Когда кластер Продукт разделен на 2 меньших агрегата, сослаться на них по необходимости можно по их идентификаторам. Ссылка на Продукт идет по его идентификатору, что экономит память и другие ресурсы и защищает от изменения. Данное явление изображено на рисунке 14.

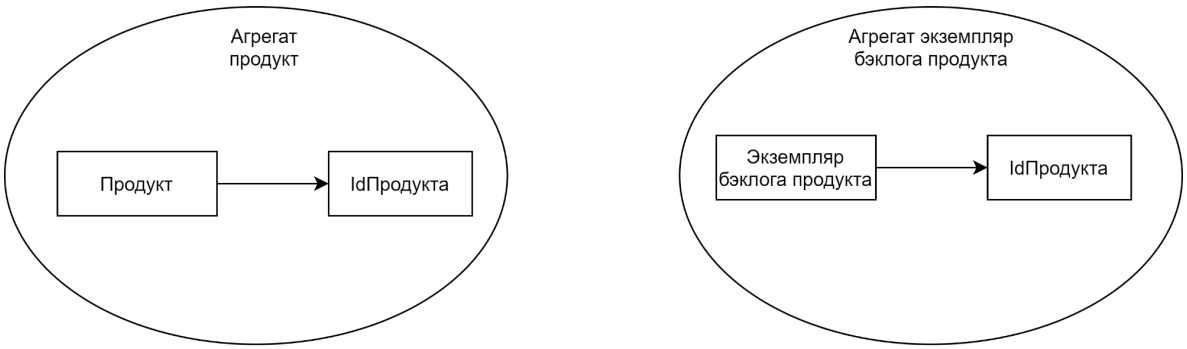


Рисунок 14 ссылка на агрегаты по идентификаторам

Четвертым и последним правилом является обновление других агрегатов руководствуясь принципом итоговой согласованности.
Важно понимать, что события предметной области издаются агрегатом. Заинтересованный ограниченный контекст подписывается на их получение. данное явление изображено на рисунке 15.

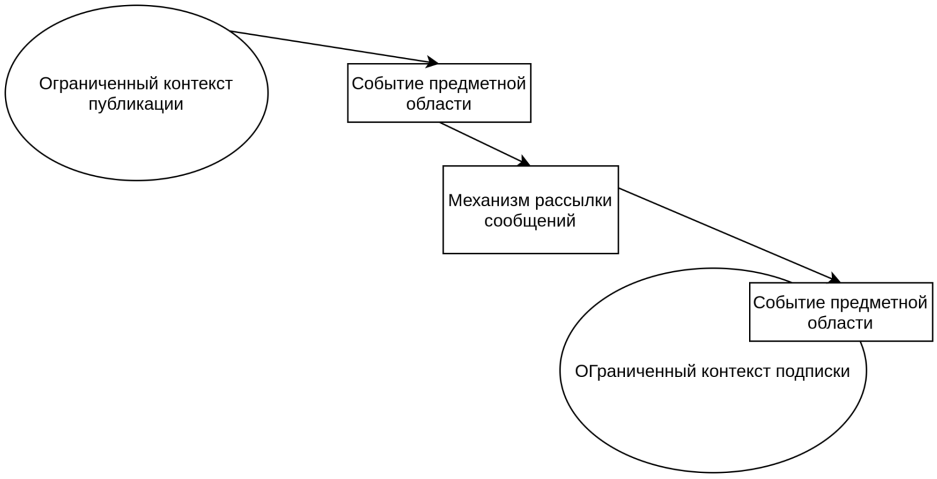


Рисунок 15 - события предметной области

Механизм передачи сообщений поставляет события предметной области заинтересованным сторонам с помощью инструментов подписки. При чем ограниченный контекст публикации и подписки может быть одним тем же ограниченным контекстом.
При создании модели разработчика подстерегает одна из самых частых ошибок при создании агрегатов - анемичная модель предметной области. Это объектно-ориентированная модель предметной области, в которой все агрегаты имеют только публичные геттеры, и сеттеры для свойств класса и не содержат никакой бизнес-логики. Проектирование анемичной модели предметной области означает формальное применение модели предметной области без реализации ни одного из ее преимуществ.
Также нельзя допускать уход бизнес-логики в прикладные службы. Это называется скрытой анемией. Делегирование бизнес-логики от служб во вспомогательные/служебные классы тоже ни к чему хорошему не приводит. Бизнес-логика должна быть в модели предметной области.
Эффективная программная модель всегда основана на ряде абстракций. Следовательно, необходимо правильно выбирать соответствующий уровень абстракции для каждой моделируемой концепции. Подчинение правилам единого языка гарантирует создание правильных абстракций. В итоге создания неправильных абстракций возникают следующие проблемы:

1. Язык программной модели не соответствует ментальной модели экспертов предметной области.

2. Уровень абстракции слишком высокий, при моделировании деталей из-за этого могут возникнуть проблемы.
3. Путь неправильных абстракций - путь создания сложной иерархии классов с соответствующими проблемами.
4. Создание неправильных абстракций - путь решения неразрешимой проблемы, которая не должна рассматриваться вообще, что ведет к созданию большего количества кода, чем требуется.
5. Часто язык неправильных абстракций отражается на пользовательском интерфейсе, который вызывает замешательство у пользователей, что является в корне неправильным.
6. Тратятся ресурсы: время и деньги, которые очень ценятся бизнесом.
7. Нельзя предвидеть будущие потребности, поэтому делать то, что описано в предыдущем пункте не стоит.

В прошлом уже упоминались события предметной области. Событие предметной области - очень важный инструмент стратегического проектирования. Однако часто в ходе тактического проектирования события предметной области схематизируются и становятся частью смыслового ядра.

Для понимания мощи событий предметной области будет рассмотрена концепция причинной согласованности. Предметная область обеспечивает причинную согласованность, если порядок ее причинно связанных операций, то есть операций, одна из которых вызывает другую, сохраняется в каждом зависящем узле распределенной системы. Это означает, что причинно связанные операции должны выполняться в определенном порядке, и, следовательно, событие не может произойти, пока не произойдет предшествующее ему событие.

Благодаря тактическому проектированию события предметной области становятся частью модели предметной области и могут публиковаться и рассылаться в ограниченном контексте. Для эффективного проектирования и реализации событий предметной области в ограниченном контексте необходимо выполнить следующие действия, описанные далее. На следующей диаграмме классов на рисунке 16 представлен минимальный интерфейс DomainEventInterface, который представляет минимальную функциональность для события предметной области. данный интерфейс имеет одно необязательное свойство OccuredOn для обозначения времени, когда произошло событие. Для некоторых языков это может быть абстрактный класс, в зависимости от типа реализации ООп парадигмы.



Рисунок 16. Диаграмма классов минимального интерфейса события предметной области.

Имена типов событий предметной области должны быть утверждением о произошедшем, то есть быть глаголом в прошедшем времени. Вот некоторые примеры из Контекста управления гибким проектированием(Scrum): ProductCreated, ReleaseScheduled, SprintScheduled, BacklogItemPlanned, BacklogItemCommited. Все сетко и ясно сообщают, что произошло в смысловом ядре.

Комбинация имен событий предметной области и их свойств должна полностью описывать то, что случилось в модели предметной области. Например для события ProductCreated диаграмма классов будет выглядеть следующим образом. Она представлена на рисунке 17. Свойства tenantId - идентифицирует арендатора-подписчика, productId - идентифицирует

уникальный создаваемый объект класса Product, name - идентифицирует имя объекта класса Product, description - идентифицирует описание объекта класса Product.

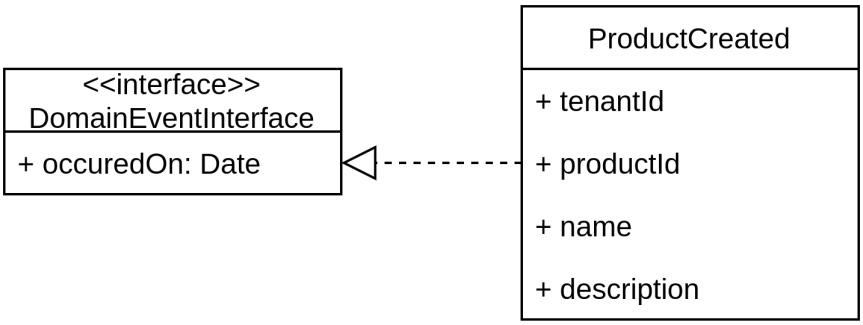


Рисунок 17. Диаграмма классов для события ProductCreated.

Событие полностью и точно сообщит всем подписчикам, что случилось в модели, то есть объект класса Product был создан для арендатора с идентификатором tenantId.

Инструментальное средство для повышения эффективности проектирования

Используя DDD, необходимо глубоко разбираться в бизнес-процессах, а затем, основываясь на полученных знаниях моделировать программное обеспечение. Это процесс изучения, экспериментирования, обсуждения, более глубокого изучения и повторного моделирования. Часто на это просто нет времени, так как руководство дает четкие урезанные сроки на моделирование, за которые качественную модель сделать сложно. Существует несколько средств повышения эффективности проектирования, которые будут рассмотрены далее. Событийный штурм — это методика быстрого проектирования, предназначенная для вовлечения экспертов предметной области и разработчиков в быстро изменяющийся процесс изучения. Методика сосредоточена на бизнесе и бизнес-процессах, а не на данных и существительных. Для применения данного нужны стикеры. Данный подход обладает следующими особенностями и преимуществами:

1. Осязательный подход. Каждый участник получает стикеры и ручку и должен внести свой вклад в изучение и решение проблемы. Каждый делает вклад в единый язык, как разработчики так и на равных эксперты предметной области.
2. Поход сосредотачивает внимание каждого на событиях и бизнес-процессе, а не реализации данных понятий.
3. Подход исключительно визуален.
4. Данный подход быстро организуем и дешев.
5. Группа совершит прорыв и получит глубокие знания.
6. Каждый получает какие-то знания.
7. Подход подразумевает идентификацию проблем, связанных с моделью и знаниями о бизнес-процессах на самых ранних этапах.
8. Событийный штурм можно использовать как для концептуального так для проектного моделирования.
9. Необязательно ограничиваться одним сеансом штурма.

Что необходимо для штурма модели? Ниже приведен подробный список простых требований:

1. Необходим привлечь к штурму нужных людей: экспертов предметной области и разработчиков.
2. Каждый должен быть непредвзятым и свободным от предубеждений.

3. Запас стикеров, очень большой запас стикеров следующих цветов: фиолетового или красного, светло-голубого, оранжевого, бледно-желтого, сиреневого и розового и других цветов.
4. Черный маркер для каждого участника.
5. Широкая стена для навешивания рулона бумаги, на который будут вешаться стикеры.

Для проведения событийного штурма нужно выполнить все этапы, описанные далее в определенной последовательности.

Первым этапом является штурм бизнес-процесса с помощью создания ряда событий предметной области на стикерах. Самый популярный цвет для событий предметной области оранжевый. Как это примерно выглядит представлено на рисунке 18.

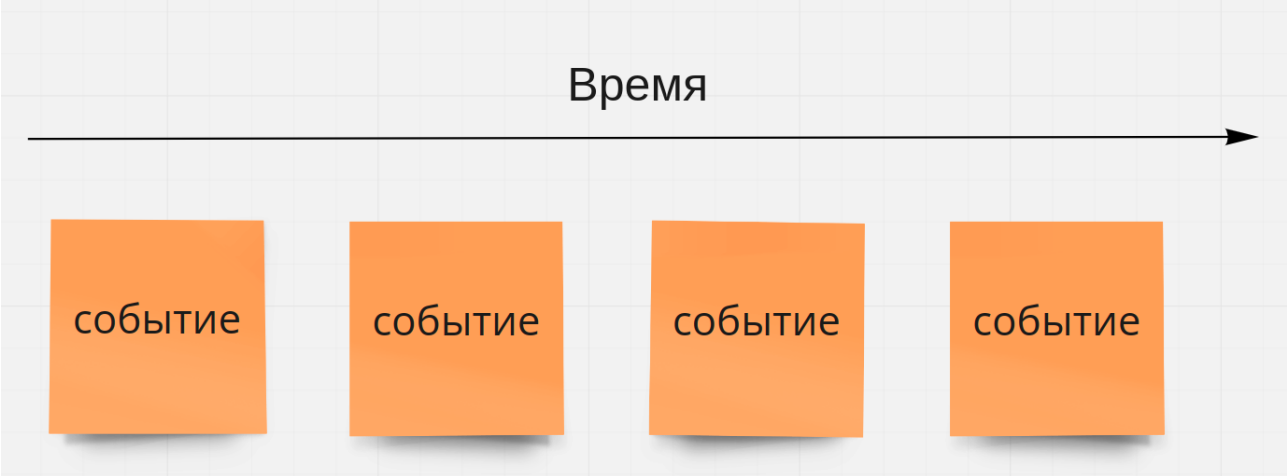


Рисунок 18. Моделирование событий предметной области.

Далее приводятся обобщенные рекомендации к данному этапу моделирования предметной событийного штурма.

1. Сосредоточение на бизнес-процессах.
2. Соблюдение правил именования событий предметной области.
3. Размещение событий в хронологическом порядке.
4. Выделение проблемных мест с описанием на красных или фиолетовых стикерах.
5. Иногда результатом событий предметной области является процесс, который необходимо запустить. Каждое событие предметной области, которое запускает процесс, должно быть зафиксировано и названо на сиреневом стикере.

Переход на следующий этап осуществляется тогда, когда обнаружена большая часть самых важных событий предметной области.

Следующий этап предполагает создание всех команд, вызывающих события предметной области. Иногда событие предметной области является результатом события, произошедшего в другой системе и поступившего в систему. Напомним, что имя команды должно быть глаголом. Основные рекомендации по созданию команд приведены ниже. А на рисунке 19 приведен пример модифицированной доски.

1. Имена команд, вызывающих соответствующее событие предметной области, на светло-голубом стикере.
2. Стикеры команд размещаются справа от событий предметной области, которые они инициируют.
3. Если есть пользовательская роль, то обязательно размещение маленького желтого стикера с упоминанием данного факта. Как это сделать представлено на рисунке 20.
4. Команда, инициирующая процесс, должена быть названа и зафиксирована на сиреневом стикере.
5. Продолжение перемещения в хронологическом порядке.

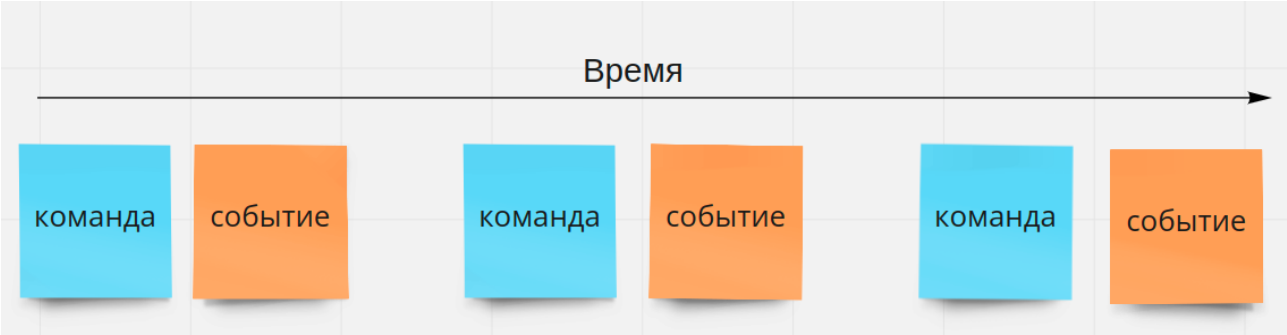


Рисунок 19. Модифицированная доска

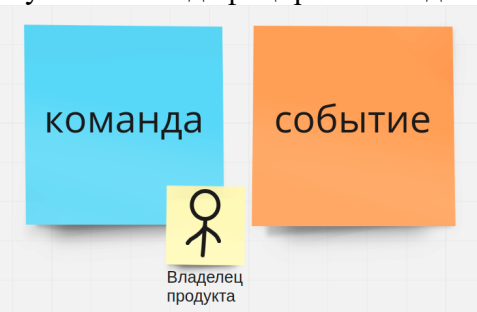


Рисунок 20. Команда из пользовательского интерфейса

Как только будет обнаружено, что все команды, связанные с событиями предметной области, которые они вызывают, готовы, то можно переходить к следующему шагу.
Третьим шагом является установление связей между сущностями/агрегатами, которым применяется команда и сущностями/агрегатами, порождающими события предметной области. Они представляют собой место хранения данных, в котором выполняются команды и генерируются события предметной области. Ниже представлены рекомендации, касающиеся моделирования агрегатов.

1. Агрегат называть сущностью или данными для облегчения экспертам предметной области жизни.
2. Использовать для агрегатов использовать светло-желтые стикеры.
3. Стикер агрегата помещать сзади пары событие предметной области и события. Это показано на рисунке 21.
4. Агрегаты можно дублировать, если они используются неоднократно.

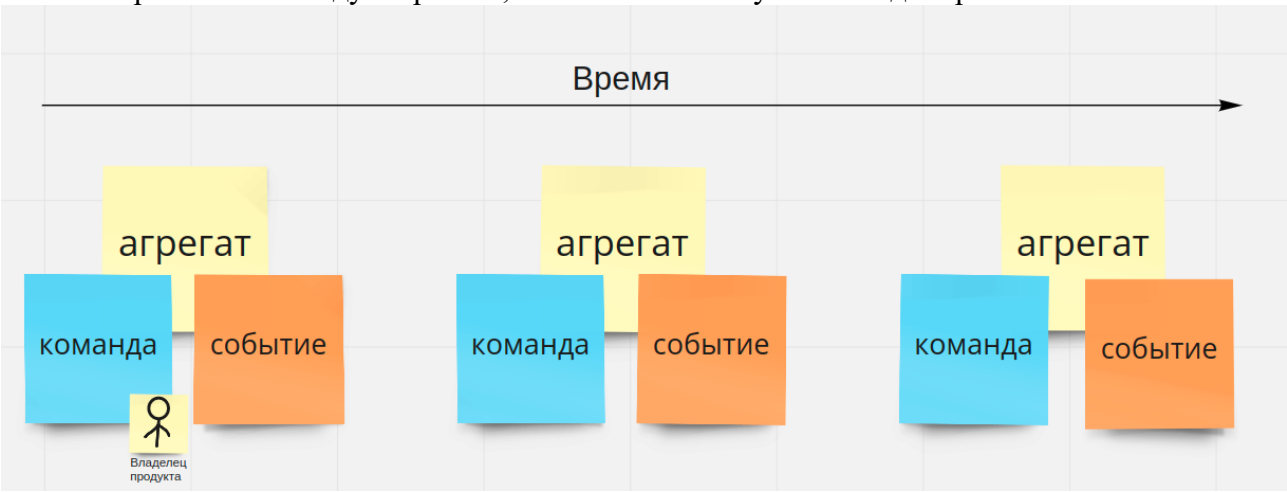


Рисунок 21. Третий этап событийного штурма

Данные этапы являются основными в событийном штурме и теперь после них можно перейти к дополнительным этапам. Данные этапы не обязательно и в данной лекции описываться не будут, так как данные этапы уже привязаны к определенной архитектуре.

Литература для изучения методологии

Для дальнейшего изучения данной методологии следует обратиться к следующей литературе:

1. Реализация методов предметно-ориентированного проектирования Вернон Вон.
2. Предметно-ориентированное проектирование (DDD). Структуризация сложных программных систем Эванс Эрик
3. Предметно-ориентированное проектирование. Самое основное Вернон Вон



Рисунок 22 Литература

Изучение следует начать с последней книги, которая является кратким и емким обзором данной методологии. Данные книги изображены на рисунке 22.

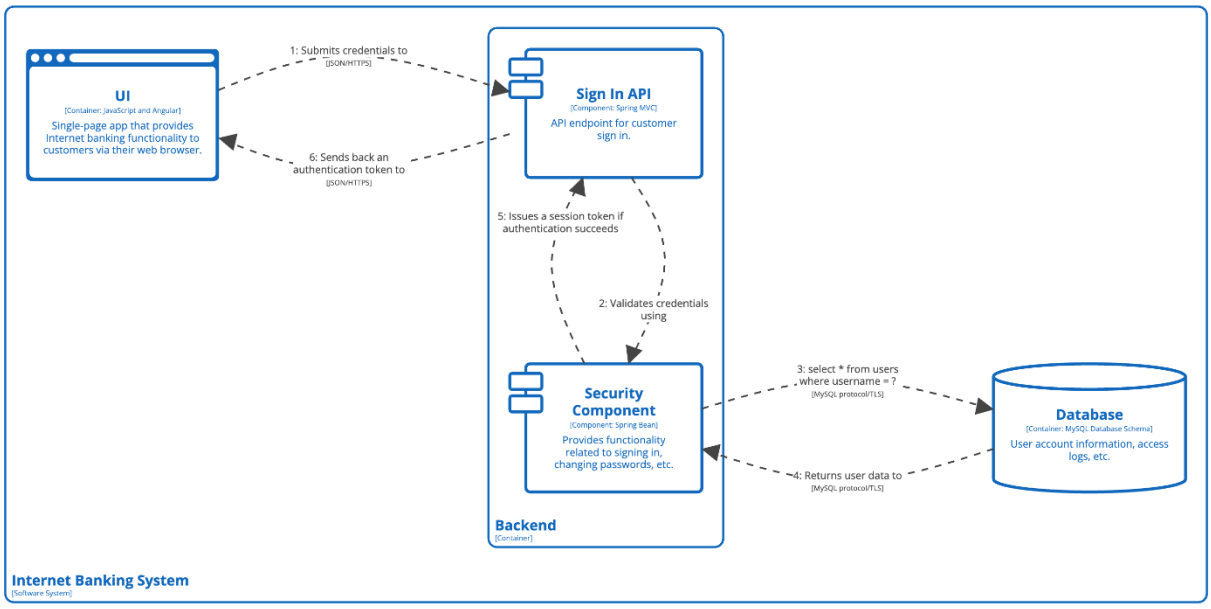
Задачи к выполнению

Доменная аналитика и карта контекстов

1. Создать Event Storming: собрать события, команды, акторов; отметить узкие места.
2. Описать поддомены: выписать способности, сгруппировать; классифицировать Core/Supporting/Generic; зафиксировать границы в 1 фразе; назначить владельцев. Визуализировать в нотации C4: **System Landscape**: узлы-контексты с тегами Core/Supporting/Generic и владельцами.
3. Сформировать контекст-карту: связи между контекстами (Partnership, Up/Downstream, Conformist, Shared Kernel, ACL, OHS/Published Language, Separate Ways) с протоколом, SLA/частотой изменений, ответственностью; риски и правила эволюции. Визуализировать в нотации C4: **System Landscape** (или **System Context** для Core): подписи на связях + легенда.
4. Зафиксировать по Ubiquitous Language: термины, определения, границы значений, источник истины; убрать омонимы; синхронизировать имена сущностей/команд/событий.

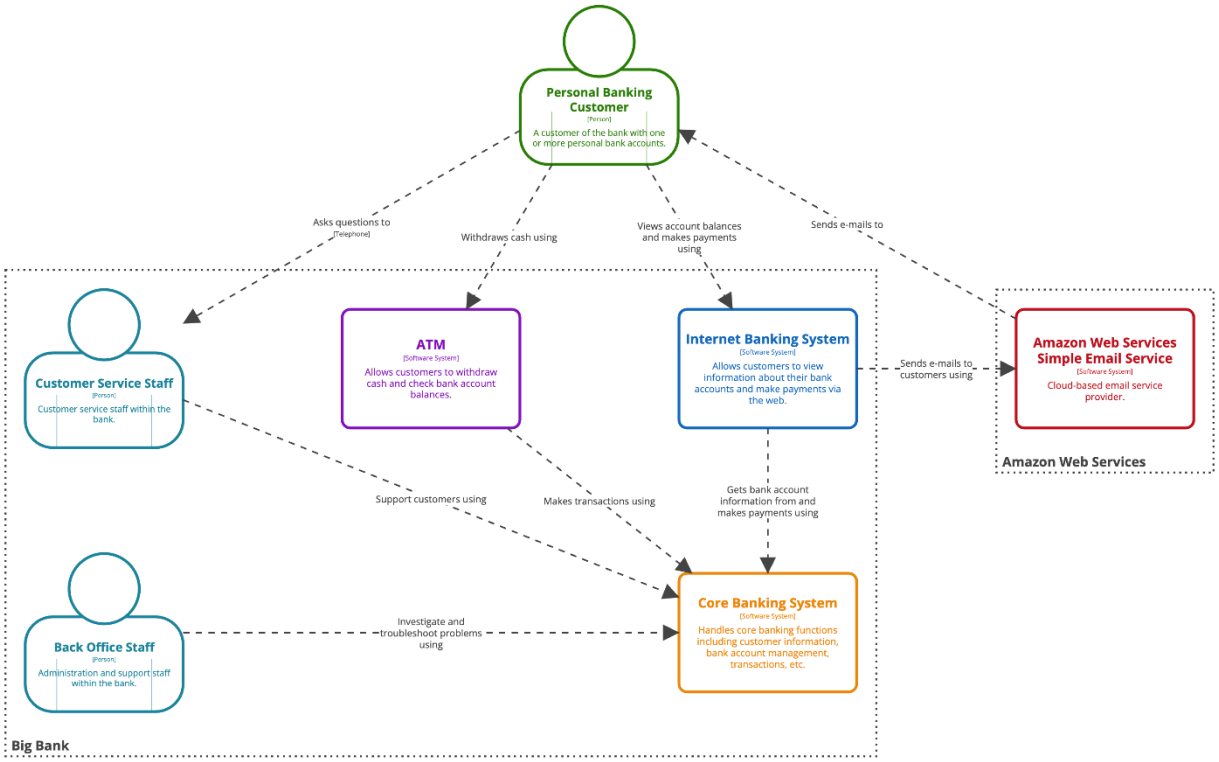
Технический дизайн и событийные потоки

1. Описать порты и адаптеры: вход (UI/API/Auth), сервис приложения, домен, выход (репозитории, брокер, внешние сервисы); зафиксировать контракты и точки расширения. Визуализировать в нотации C4: **Container** + подписи портов/контрактов.
2. Зафиксировать агрегаты: кандидаты по атомарности; корень, сущности, объекты-значения; инварианты; доступ только через корень; связи между агрегатами — по ID; команды и публикуемые события. Визуализировать в нотации C4: **Component** внутри Domain-контейнера (Aggregate Roots, Policies, Repositories) + краткие аннотации инвариантов.
3. Сформулировать доменные события и причинные цепочки: имя, атрибуты, издатель (агрегат), подписчики, последствия; границы итоговой согласованности, требования к доставке. Визуализировать в нотации C4: **Dynamic** (2–3 ключевых сценария) + на **Container/Component** отметить publishes/consumes и брокер/темы.



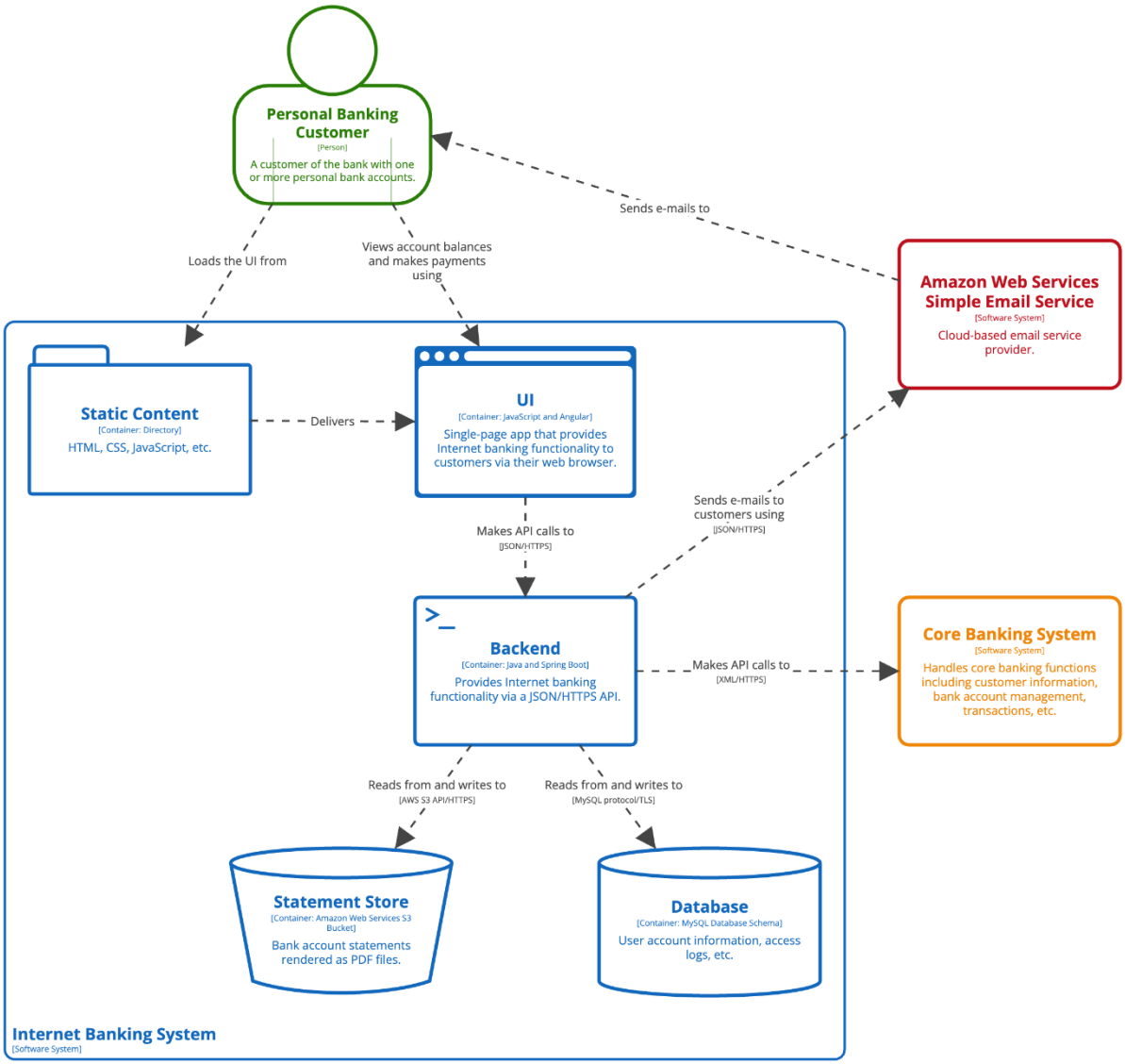
Dynamic View: Internet Banking System - Backend

Summarises how the sign in feature works in the single-page application | Simon Brown | c4model.com | License: CC BY 4.0



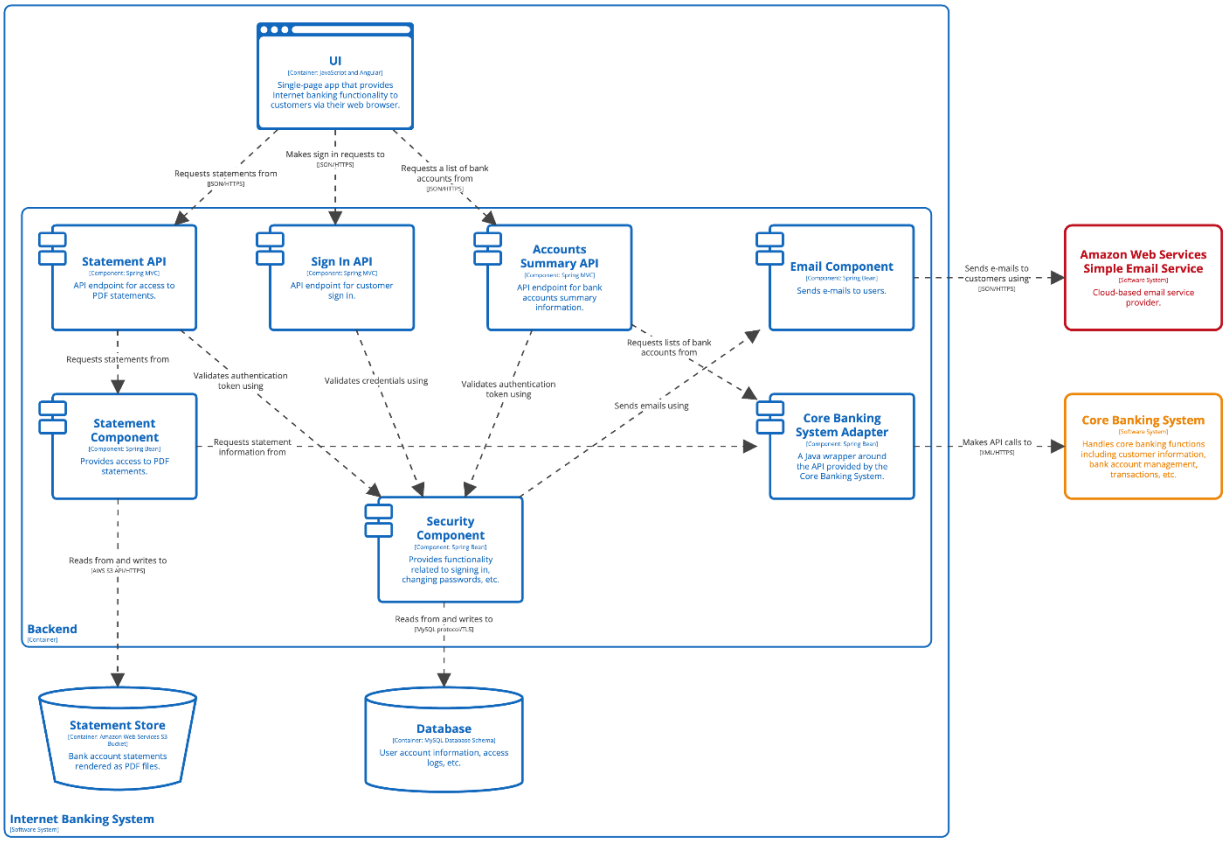
System Landscape View

A partial system landscape diagram for a fictional bank | Simon Brown | c4model.com | License: CC BY 4.0



Container View: Internet Banking System

The container diagram for the Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0



Component View: Internet Banking System - Backend

The component diagram for the Internet Banking System Backend | Simon Brown | c4model.com | License: CC BY 4.0

https://c4model.com/tooling/diagramming